

Biyomedikal İşaret ve Görüntü İşleme Ödev 5: Gray Scale Histogram

Muhammed Kayra Bulut - 25501805

Kasım 2025

1 Giriş

Dijital görüntü işlemede histogram, bir görüntüdeki piksel yoğunluk değerlerinin dağılımını gösteren temel bir analiz aracıdır. Bir görüntünün histogramı, görüntünün kontrastı, parlaklığı ve dinamik aralığı hakkında önemli bilgiler sunar. Bu ödev kapsamında iki temel çalışma gerçekleştirilmiştir.

İlk aşamada (Bölüm A), gri seviye (gray-scale) görüntülerin histogramları ve logaritmik histogramları çıkartılmıştır. Logaritmik ölçeklendirme, frekansı düşük olan piksel değerlerinin daha görünür hale gelmesini sağlayarak, standart histogramda gözden kaçabilecek detayların analiz edilmesine olanak tanır. Bu işlem üç farklı örnek görüntü üzerinde uygulanmış ve normalize edilerek aynı grafik üzerinde karşılaştırılmıştır.

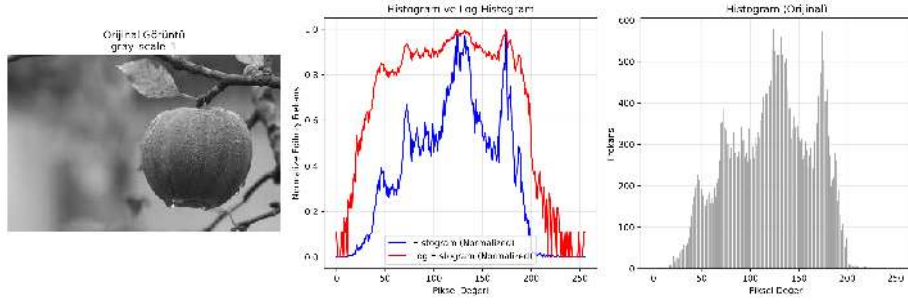
İkinci aşamada (Bölüm B), düşük kontrastlı bir görüntünün histogram eşitleme (histogram equalization) yöntemiyle iyileştirilmesi hedeflenmiştir. Standart eşitleme işleminin ötesinde, çıkış histogramı belirli piksel aralıklarına $[0, 130]$, $[100, 170]$ ve $[130, 255]$ sıkıştırılarak, görüntünün belirli parlaklık bölgelerindeki davranışı ve Kümülatif Dağılım Fonksiyonu'nun (CDF) değişimi incelenmiştir.

2 Sonuçlar

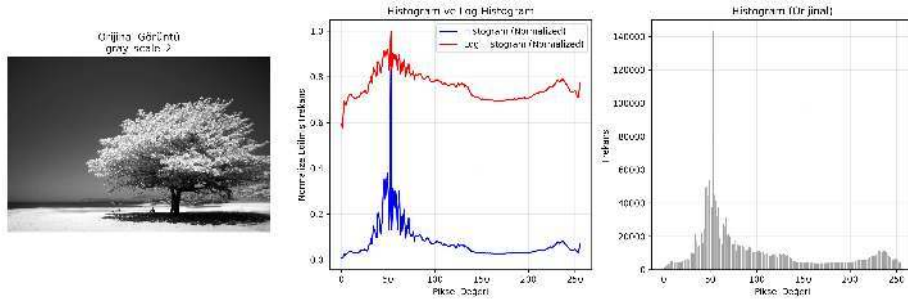
2.1 Bölüm A: Histogram ve Log-Histogram Analizi

Bu bölümde, seçilen üç farklı gri seviye görüntü için piksel yoğunluk dağılımları analiz edilmiştir. Grafikler, orijinal histogram ile normalize edilmiş logaritmik histogramı aynı eksen üzerinde göstermektedir.

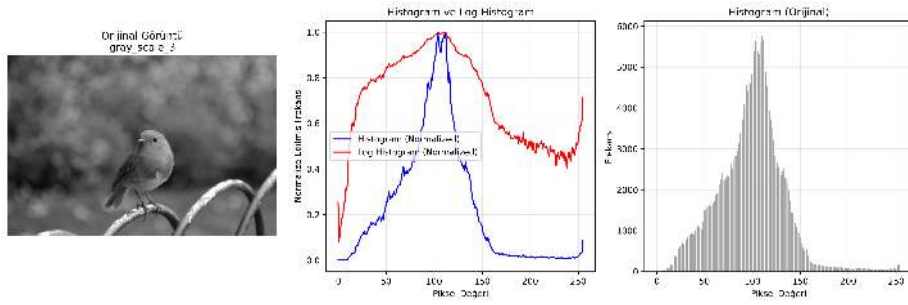
Şekil 1'de bir elma görseli, Şekil 2'de bir ağaç manzarası ve Şekil 3'te bir kuş görseli kullanılmıştır. Mavi çizgiler normalize edilmiş standart histogramı, kırmızı çizgiler ise logaritmik histogramı temsil etmektedir. Logaritmik gösterim sayesinde, özellikle piksel sayısının az olduğu koyu ve çok parlak bölgelerdeki dağılım detayları daha net gözlemlenebilmiştir.



Şekil 1: Örnek 1: Elma görselinin orijinal hali, normalize histogram ve log-histogram grafikleri.



Şekil 2: Örnek 2: Manzara görselinin histogram analizi.

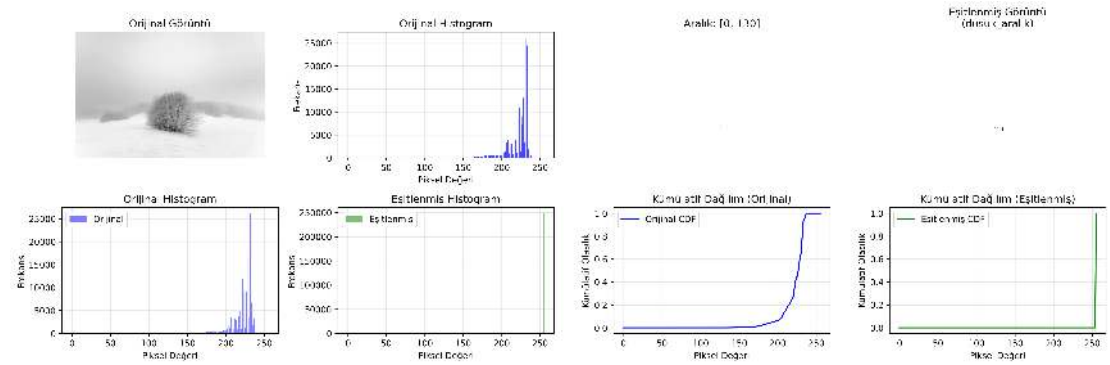


Şekil 3: Örnek 3: Kuş görselinin histogram ve log-histogram analizi.

2.2 Bölüm B: Belirli Aralıklarda Histogram Eşitleme

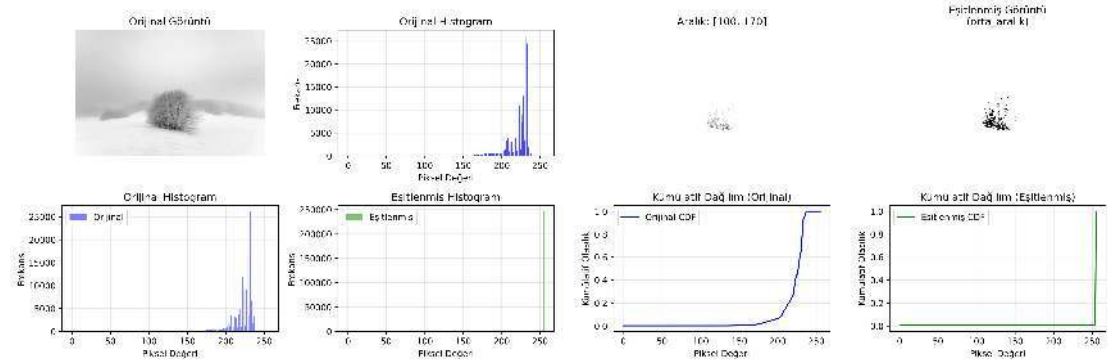
Bu bölümde, düşük kontrastlı ve ağırlıklı olarak açık tonlara (yüksek piksel değerlerine) sahip bir kış manzarası görüntüsü üzerinde çalışılmıştır. Orijinal histogramın dar bir alana sıkıştığı görülmektedir. Görüntü, üç farklı hedef aralığa göre yeniden örneklenmiş ve histogram eşitleme uygulanmıştır.

İlk olarak, piksel değerleri $[0, 130]$ aralığına (düşük aralık) çekilmiştir. Şekil 4'te görüldüğü üzere, görüntü koyulaşmış ve histogram sol tarafa (karanlık bölgeye) yığılmıştır. Eşitlenmiş CDF (yeşil çizgi), bu dar aralıkta lineer bir artış göstermektedir.



Şekil 4: Düşük Aralık $[0, 130]$ için histogram eşitleme sonuçları.

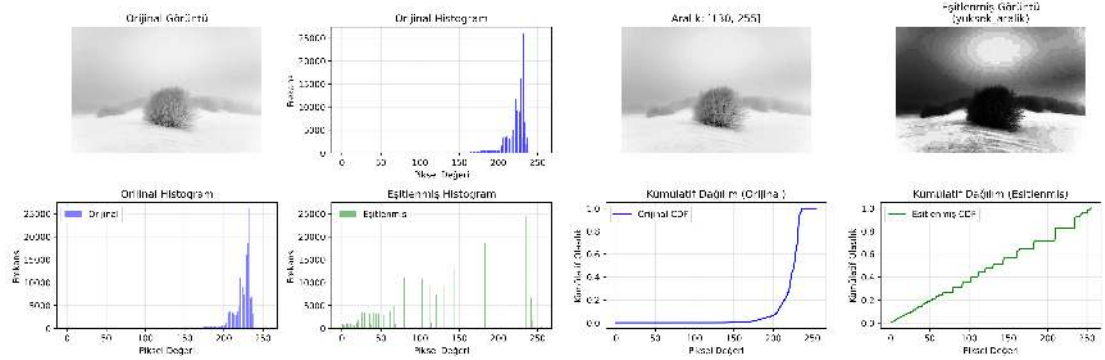
İkinci denemede, hedef aralık $[100, 170]$ (orta aralık) olarak belirlenmiştir. Şekil 5'da görüldüğü gibi, görüntüdeki kontrast orta tonlarda toplanmış, çok karanlık ve çok parlak alanlar elimine edilmiştir.



Şekil 5: Orta Aralık $[100, 170]$ için histogram eşitleme sonuçları.

Son olarak, görüntü $[130, 255]$ (yüksek aralık) değerlerine eşitlenmiştir. Şekil 6'te görüldüğü üzere, bu işlem orijinal görüntünün karakterine en yakın

sonucu vermiş ancak kontrastı yüksek parlaklık değerlerine yayarak detayların (örneğin kardaki dokuların) daha belirgin hale gelmesini sağlamıştır.



Şekil 6: Yüksek Aralık [130, 255] için histogram eşitleme sonuçları.

3 Kod

Kod aşağıda verilmiştir:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

# Klasör yapısını oluştur
os.makedirs('dest', exist_ok=True)

def calculate_and_plot_histogram(image_path, output_name):
    """
    Bir gray-level görüntünün histogramini ve log
    histogramini hesaplayıp kaydeder
    """
    # Görüntüyü oku
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

    if img is None:
        print(f"Hata: {image_path} dosyası okunamadı!")
        return

    # Histogram hesapla
    hist = cv2.calcHist([img], [0], None, [256], [0, 256])
    hist = hist.flatten()

    # Log histogram hesapla (0 değerlerini onlemek için +1
    ekliyoruz)
```

```

log_hist = np.log(hist + 1)

# Normalize et (ayni grafikte gosterme icin)
hist_normalized = hist / hist.max()
log_hist_normalized = log_hist / log_hist.max()

# Grafik olustur
plt.figure(figsize=(15, 5))

# Orijinal goruntu
plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray')
plt.title(f'Orijinal Goruntu\n{output_name}')
plt.axis('off')

# Histogram ve Log Histogram
plt.subplot(1, 3, 2)
plt.plot(hist_normalized, color='blue', label='Histogram
(Normalized)')
plt.plot(log_hist_normalized, color='red', label='Log
Histogram (Normalized)')
plt.title('Histogram ve Log Histogram')
plt.xlabel('Piksel Degeri')
plt.ylabel('Normalize Edilmis Frekans')
plt.legend()
plt.grid(True, alpha=0.3)

# Sadece histogram
plt.subplot(1, 3, 3)
plt.bar(range(256), hist, color='gray', alpha=0.7)
plt.title('Histogram (Orijinal)')
plt.xlabel('Piksel Degeri')
plt.ylabel('Frekans')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(f'dest/{output_name}_histogram.png', dpi
=300, bbox_inches='tight')
plt.close()

def histogram_equalization_custom_range(image_path, min_val,
max_val, range_name):
    """
    Belirli bir aralik icin histogram esitleme yapar
    """
    # Goruntuyu oku
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

    if img is None:

```

```

        print(f"Hata: {image_path} dosyasi okunamadi!")
        return

# Orijinal histogram
hist_original = cv2.calcHist([img], [0], None, [256],
                             [0, 256]).flatten()

# Belirli aralikta histogram esitleme
img_clipped = np.clip(img, min_val, max_val)

# Araligi 0-255'e normalize et
img_normalized = ((img_clipped - min_val) / (max_val -
                                             min_val) * 255).astype(np.uint8)

# Histogram esitleme uygula
img_equalized = cv2.equalizeHist(img_normalized)

# Histogram hesapla
hist_equalized = cv2.calcHist([img_equalized], [0], None
                              , [256], [0, 256]).flatten()

# Grafik olustur
plt.figure(figsize=(18, 6))

# Orijinal goruntu
plt.subplot(2, 4, 1)
plt.imshow(img, cmap='gray')
plt.title('Orijinal Goruntu')
plt.axis('off')

# Orijinal histogram
plt.subplot(2, 4, 2)
plt.bar(range(256), hist_original, color='blue', alpha
        =0.7)
plt.title('Orijinal Histogram')
plt.xlabel('Piksel Degeri')
plt.ylabel('Frekans')
plt.grid(True, alpha=0.3)

# Kirpilmis goruntu
plt.subplot(2, 4, 3)
plt.imshow(img_normalized, cmap='gray')
plt.title(f'Aralik: [{min_val}, {max_val}]')
plt.axis('off')

# Esitlenmis goruntu
plt.subplot(2, 4, 4)
plt.imshow(img_equalized, cmap='gray')
plt.title(f'Esitlenmis Goruntu\n({range_name})')
plt.axis('off')

```

```

# Karsilastirmali histogramlar
plt.subplot(2, 4, 5)
plt.bar(range(256), hist_original, color='blue', alpha
        =0.5, label='Orijinal')
plt.title('Orijinal Histogram')
plt.xlabel('Piksel Degeri')
plt.ylabel('Frekans')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(2, 4, 6)
plt.bar(range(256), hist_equalized, color='green', alpha
        =0.5, label='Esitlenmis')
plt.title('Esitlenmis Histogram')
plt.xlabel('Piksel Degeri')
plt.ylabel('Frekans')
plt.legend()
plt.grid(True, alpha=0.3)

# Kumulatif dagilim fonksiyonlari
plt.subplot(2, 4, 7)
cdf_original = np.cumsum(hist_original)
cdf_original = cdf_original / cdf_original.max()
plt.plot(cdf_original, color='blue', label='Orijinal CDF
        ')
plt.title('Kumulatif Dagilim (Orijinal)')
plt.xlabel('Piksel Degeri')
plt.ylabel('Kumulatif Olasilik')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(2, 4, 8)
cdf_equalized = np.cumsum(hist_equalized)
cdf_equalized = cdf_equalized / cdf_equalized.max()
plt.plot(cdf_equalized, color='green', label='Esitlenmis
        CDF')
plt.title('Kumulatif Dagilim (Esitlenmis)')
plt.xlabel('Piksel Degeri')
plt.ylabel('Kumulatif Olasilik')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(f'dest/low-contrast-equalized-{range_name}.
        png', dpi=300, bbox_inches='tight')
plt.close()

# Esitlenmis goruntuyu kaydet
cv2.imwrite(f'dest/low-contrast-equalized-{range_name}-

```

```

        image.jpg', img_equalized)

# Ana program
if __name__ == "__main__":
    print("=" * 60)
    print("oDEV 5: Histogram Analizi ve Esitleme")
    print("=" * 60)

    # A) Gray-scale görüntülerin histogramlarını çıkar
    print("\n[A] Gray-scale görüntülerin histogramları
          hesaplanıyor...\n")

    gray_images = [
        'src/gray-scale-1.jpg',
        'src/gray-scale-2.jpg',
        'src/gray-scale-3.jpg'
    ]

    for i, img_path in enumerate(gray_images, 1):
        if os.path.exists(img_path):
            calculate_and_plot_histogram(img_path, f'gray-
            scale_{i}')

    # B) Düşük kontrastlı görüntü için histogram esitleme
    print("\n" + "=" * 60)
    print("[B] Düşük kontrastlı görüntü için histogram
          esitleme yapılıyor...\n")

    low_contrast_path = 'src/low-contrast.jpg'

    if os.path.exists(low_contrast_path):
        # Düşük aralık (0-130)
        print("\n1. Düşük Kontrast Aralığı (0-130):")
        histogram_equalization_custom_range(low_contrast_
            path, 0, 130, 'dusuk-aralik')

        # Orta aralık (100-170)
        print("\n2. Orta Kontrast Aralığı (100-170):")
        histogram_equalization_custom_range(low_contrast_
            path, 100, 170, 'orta-aralik')

        # Yüksek aralık (170-255)
        print("\n3. Yüksek Kontrast Aralığı (170-255):")
        histogram_equalization_custom_range(low_contrast_
            path, 130, 255, 'yuksek-aralik')

    print("\n" + "=" * 60)

```

Listing 1: main.py